



Last updated on **Sep 11, 2026**

Unity Catalog privileges reference

This page is a reference for Unity Catalog privileges and the securable objects they apply to.

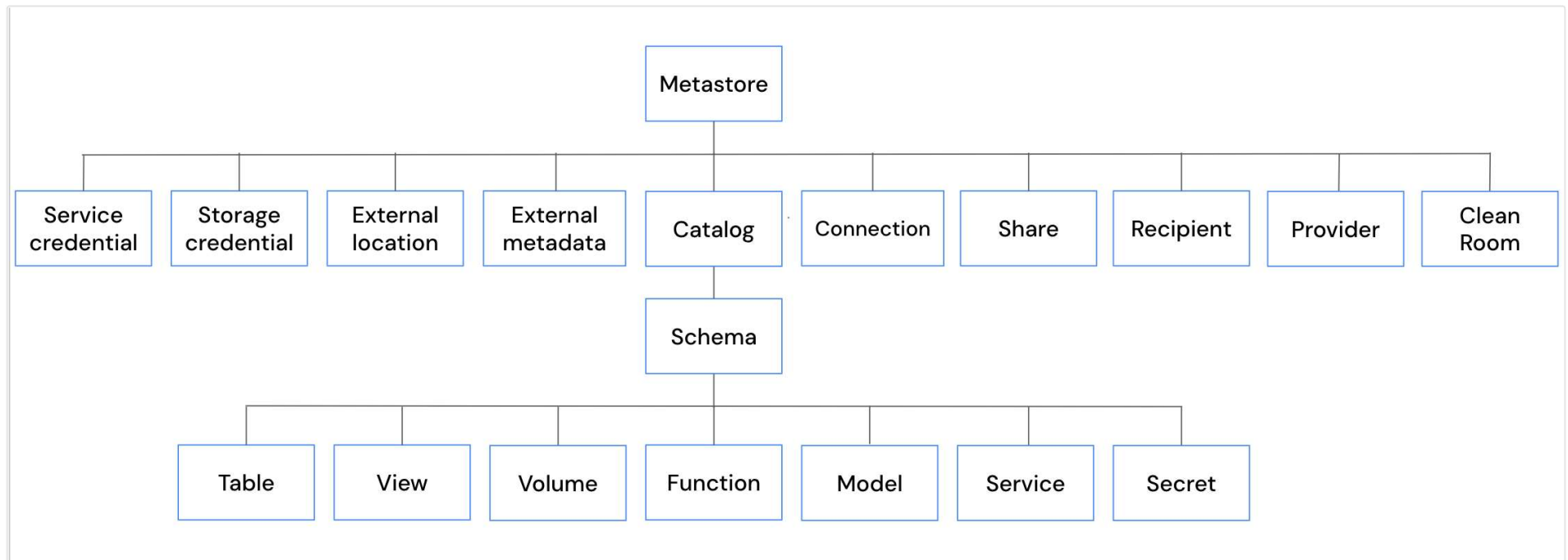
For detailed descriptions of each securable object type, see [Unity Catalog securable objects reference](#). To learn how to grant privileges in Unity Catalog, see [Show, grant, and revoke privileges](#).

NOTE

This page refers to the Unity Catalog privileges and inheritance model in Privilege Model version 1.0. If you created your Unity Catalog metastore during the public preview (before August 25, 2022), you might be on an earlier privilege model that doesn't support the current inheritance model. You can upgrade to Privilege Model version 1.0 to get [privilege inheritance](#). See [Upgrade to privilege inheritance](#).

Securable objects in Unity Catalog

A securable object is an object defined in the Unity Catalog [metastore](#) on which privileges can be granted to a principal (user, service principal, or group). Securable objects in Unity Catalog are hierarchical, from the [metastore](#) at the top, through [catalogs](#) and [schemas](#), down to the data objects they contain ([tables](#), [views](#), [volumes](#), [functions](#), and [models](#)). Additional securable objects govern access to external storage, external services, and OpenSharing.



For detailed descriptions of each securable object type, see [Unity Catalog securable objects reference](#).

What privileges apply to each securable object?

The following table lists the privileges that apply to each securable object in Unity Catalog. To learn how to grant privileges in Unity Catalog, see [Show, grant, and revoke privileges](#).

Securable	Privileges
Metastore	CREATE CATALOG, CREATE CLEAN ROOM, CREATE CONNECTION, CREATE EXTERNAL LOCATION, CREATE EXTERNAL METADATA, CREATE PROVIDER, CREATE RECIPIENT, CREATE SHARE, CREATE SERVICE CREDENTIAL, CREATE STORAGE CREDENTIAL, MANAGE ALLOWLIST, READ METADATA, SET SHARE PERMISSION, USE MARKETPLACE ASSETS, USE PROVIDER, USE RECIPIENT, USE SHARE

Securable	Privileges
	When granted on the metastore, <code>READ METADATA</code> inherits down to all objects in the metastore. This is unlike other metastore-level privileges, which don't follow privilege inheritance.
Catalog	<code>ALL PRIVILEGES</code>, <code>APPLY TAG</code>, <code>BROWSE</code>, <code>CREATE SCHEMA</code>, <code>USE CATALOG</code> All users have <code>USE CATALOG</code> on the <code>main</code> catalog by default. The following privileges apply to securable objects in a catalog. You can grant these privileges at the catalog level to apply them to current and future objects in the catalog. <code>CREATE FEATURE</code>, <code>CREATE FUNCTION</code>, <code>CREATE SECRET</code>, <code>CREATE TABLE</code>, <code>CREATE MATERIALIZED VIEW</code>, <code>CREATE MODEL</code>, <code>CREATE SERVICE</code>, <code>CREATE VOLUME</code>, <code>DELETE</code>, <code>EXTERNAL USE SCHEMA</code>, <code>INSERT</code>, <code>READ FEATURE</code>, <code>READ METADATA</code>, <code>READ SECRET</code>, <code>READ VOLUME</code>, <code>REFERENCE SECRET</code>, <code>REFRESH</code>, <code>UPDATE</code>, <code>WRITE SECRET</code>, <code>WRITE VOLUME</code>, <code>EXECUTE</code>, <code>MANAGE</code>, <code>MODIFY</code>, <code>SELECT</code>, <code>USE SCHEMA</code>
Schema	<code>ALL PRIVILEGES</code>, <code>APPLY TAG</code>, <code>CREATE FEATURE</code>, <code>CREATE FUNCTION</code>, <code>CREATE SECRET</code>, <code>CREATE TABLE</code>, <code>CREATE MODEL</code>, <code>CREATE SERVICE</code>, <code>CREATE VOLUME</code>, <code>CREATE MATERIALIZED VIEW</code>, <code>MANAGE</code>, <code>READ METADATA</code>, <code>EXTERNAL USE SCHEMA</code>, <code>USE SCHEMA</code> The following privileges apply to securable objects within a schema. You can grant these privileges at the schema level to apply them to current and future objects within the schema. <code>DELETE</code>, <code>EXECUTE</code>, <code>INSERT</code>, <code>MODIFY</code>, <code>READ FEATURE</code>, <code>READ SECRET</code>, <code>READ VOLUME</code>, <code>REFERENCE SECRET</code>, <code>REFRESH</code>, <code>SELECT</code>, <code>UPDATE</code>, <code>WRITE SECRET</code>, <code>WRITE VOLUME</code>
Table	<code>ALL PRIVILEGES</code>, <code>APPLY TAG</code>, <code>DELETE</code>, <code>INSERT</code>, <code>MANAGE</code>, <code>MODIFY</code>, <code>READ METADATA</code>, <code>SELECT</code>, <code>UPDATE</code>

Securable	Privileges
	INSERT, UPDATE, and DELETE are currently in Beta . See Fine-grained DML privileges .
Materialized view	ALL PRIVILEGES, APPLY TAG, MANAGE, READ METADATA, REFRESH, SELECT
View	ALL PRIVILEGES, APPLY TAG, MANAGE, READ METADATA, SELECT
Volume	ALL PRIVILEGES, APPLY TAG, MANAGE, READ METADATA, READ VOLUME, WRITE VOLUME
Secret	ALL PRIVILEGES, MANAGE, READ METADATA, READ SECRET, REFERENCE SECRET, WRITE SECRET CREATE SECRET is granted at the catalog or schema level. BROWSE does not apply to secrets.
Feature	ALL PRIVILEGES, MANAGE, READ FEATURE, READ METADATA
External location	ALL PRIVILEGES, BROWSE, CREATE EXTERNAL TABLE, CREATE EXTERNAL VOLUME, CREATE FOREIGN SECURABLE, CREATE MANAGED STORAGE, EXTERNAL USE LOCATION, MANAGE, READ FILES, READ METADATA, WRITE FILES
External metadata	ALL PRIVILEGES, APPLY TAG, BROWSE, MANAGE, MODIFY, READ METADATA
Service credential	ALL PRIVILEGES, ACCESS, CREATE CONNECTION, MANAGE, READ METADATA
Storage credential	ALL PRIVILEGES, CREATE EXTERNAL LOCATION, CREATE EXTERNAL TABLE, MANAGE, READ FILES, READ METADATA, WRITE FILES

Securable	Privileges
Connection	ALL PRIVILEGES, CREATE FOREIGN CATALOG, MANAGE, READ METADATA, USE CONNECTION
Function	ALL PRIVILEGES, APPLY TAG (models only), CREATE MODEL VERSION (models only), EXECUTE, MANAGE, READ METADATA
Model	Registered models are a type of function .
Service	ALL PRIVILEGES, APPLY TAG, EXECUTE, MANAGE, READ METADATA
Share	SELECT (Can be granted to RECIPIENT)
Recipient	None
Provider	None
Clean room	ALL PRIVILEGES, BROWSE, EXECUTE CLEAN ROOM TASK, MANAGE, MODIFY CLEAN ROOM, READ METADATA

Overview of privileges in Unity Catalog

The following table summarizes the capability each Unity Catalog privilege grants. For full descriptions, see [Detailed Unity Catalog privileges reference](#).

Privilege	Allows you to
ACCESS	Use a service credential to access an external service.
ALL PRIVILEGES	Perform all capabilities on an object and its children. <code>ALL PRIVILEGES</code> does not include the <code>EXTERNAL USE SCHEMA</code> , <code>EXTERNAL USE LOCATION</code> , <code>MANAGE</code> , or <code>READ METADATA</code> privileges.
APPLY TAG	Add and edit tags on an object. For tables and views , also enables column tagging. For registered models , also enables version tagging.
BROWSE	Discover objects, view their metadata, and request access to them without needing <code>USE CATALOG</code> or <code>USE SCHEMA</code> . Granted at the catalog level.
CREATE CATALOG	Create a catalog in the metastore .
CREATE CLEAN ROOM	Create a clean room for multi-party collaboration without sharing underlying data.
CREATE CONNECTION	Create a connection to an external database for Lakehouse Federation.
CREATE EXTERNAL LOCATION	Create an external location that associates a cloud storage path with a storage credential . Required on both the metastore and the storage credential.
CREATE EXTERNAL METADATA	Create external metadata objects for use in custom data lineage configurations.
CREATE EXTERNAL TABLE	Create external tables at a specific cloud storage path using an external location or storage credential .

Privilege	Allows you to
CREATE EXTERNAL VOLUME	Create external volumes using an external location .
CREATE FEATURE	Create a feature in a schema .
CREATE FOREIGN CATALOG	Create foreign catalogs using a Lakehouse Federation connection .
CREATE FOREIGN SECURABLE	Specify authorized cloud storage paths when creating a foreign catalog .
CREATE FUNCTION	Create functions in a schema .
CREATE MANAGED STORAGE	Set a custom storage location for managed tables at the catalog or schema level, overriding the metastore default.
CREATE MATERIALIZED VIEW	Create materialized views in a schema .
CREATE MODEL	Create MLflow registered models in a schema .
CREATE MODEL VERSION	Register a new version of an existing MLflow registered model .
CREATE PROVIDER	Create an OpenSharing provider object in the metastore .

Privilege	Allows you to
CREATE RECIPIENT	Create an OpenSharing recipient object in the metastore .
CREATE SCHEMA	Create a schema in a catalog .
CREATE SECRET	Create a secret in a schema . Granted at the catalog or schema level.
CREATE SERVICE	Create a service (model service, model provider service, MCP service, or agent service) in a schema .
CREATE SERVICE CREDENTIAL	Create a service credential in the metastore .
CREATE SHARE	Create an OpenSharing share in the metastore .
CREATE STORAGE CREDENTIAL	Create a storage credential in the metastore .
CREATE TABLE	Create tables or views in a schema .
CREATE VOLUME	Create volumes in a schema .
DELETE	Delete data from a table .
EXECUTE	Invoke a function or load a registered model for inference. Also lets you view function definitions and model metadata.

Privilege	Allows you to
EXECUTE CLEAN ROOM TASK	Run notebooks and view details in a clean room .
EXTERNAL USE LOCATION	Obtain a temporary credential to access a Unity Catalog external location from an external processing engine.
EXTERNAL USE SCHEMA	Obtain a temporary credential to access Unity Catalog tables from an external processing engine via open APIs or Iceberg REST APIs.
INSERT	Add new data to a table .
MANAGE	Manage privileges, transfer ownership, rename, and drop an object.
MANAGE ACCESS CONTROL	Manage privileges, ownership, and access control policies on an object. A child of <code>MANAGE</code> . This privilege can be denied through an ABAC DENY policy , but not granted.
MANAGE ALLOWLIST	Add or modify allowed paths for init scripts, JARs, and Maven coordinates on Unity Catalog-enabled clusters.
MODIFY	Insert, update, and delete data in a table .
MODIFY CLEAN ROOM	Update a clean room 's data assets, notebooks, and comments.
READ FEATURE	Read a feature 's metadata and materialized data, and use it for model inference and training.
	

Privilege	Allows you to
READ FILES	Read files directly from a cloud storage path configured as an external location .
READ METADATA	View all owner-visible metadata for an object, including permissions, row filters, column masks, and ABAC policies, without the ability to modify it or read its data. A child of <code>MANAGE</code> .
READ SECRET	Retrieve a secret value from a notebook or job.
READ VOLUME	Read files and directories stored inside a volume .
REFERENCE SECRET	Reference a secret from a Unity Catalog object without giving the user access to the value.
REFRESH	Manually trigger a refresh of a materialized view .
SELECT	Query data from a table , view , materialized view , or OpenSharing share .
SET SHARE PERMISSION	Grant an OpenSharing recipient access to a share .
UPDATE	Modify existing data in a table .
USE CATALOG	Required to interact with any object inside a catalog . Does not itself grant access to data.
USE CONNECTION	List and view connection details. Required to use <code>remote_query</code> against an external database.

Privilege	Allows you to
USE MARKETPLACE ASSETS	Get or request access to data products shared in Databricks Marketplace.
USE PROVIDER	View OpenSharing providers in the metastore and mount shared catalogs as a recipient user.
USE RECIPIENT	View OpenSharing recipients and their shares as a provider user.
USE SCHEMA	Required to interact with any object inside a schema . Does not itself grant access to data.
USE SHARE	View OpenSharing shares and their assets as a provider user.
WRITE FILES	Write files directly to a cloud storage path configured as an external location .
WRITE SECRET	Update a secret value.
WRITE VOLUME	Add, modify, or delete files inside a volume .

Detailed Unity Catalog privileges reference

This section provides details about the privileges that apply generally to Unity Catalog. To learn how to grant privileges in Unity Catalog, see [Show, grant, and revoke privileges](#).

ACCESS



- **Applicable object types:** SERVICE CREDENTIAL

Allows a user to use a [service credential](#) to access external services.

ALL PRIVILEGES

- **Applicable object types:** CONNECTION, EXTERNAL LOCATION, EXTERNAL METADATA, FEATURE, FUNCTION (including models), MATERIALIZED VIEW, SERVICE CREDENTIAL, STORAGE CREDENTIAL, TABLE, VIEW, VOLUME
- **Applicable container objects:** SCHEMA, CATALOG

`ALL PRIVILEGES` is a special privilege that allows a user to perform all capabilities on the securable object and its child objects. `ALL PRIVILEGES` *implies* all applicable privileges for a specific object type, and Databricks doesn't explicitly grant each individual privilege.

The following table describes what `ALL PRIVILEGES` implies for data objects in the Unity Catalog hierarchy:

Object type	Implied privileges
Catalog	Implies all catalog-level privileges*
Schema	Implies all schema-level privileges*
Table	Implies the ability to perform <code>SELECT</code> , <code>MODIFY</code> , and <code>APPLY TAG</code>
View	Implies the ability to perform <code>SELECT</code> and <code>APPLY TAG</code>
Materialized view	Implies the ability to perform <code>SELECT</code> , <code>REFRESH</code> , and <code>APPLY TAG</code>

Object type	Implied privileges
Volume	Implies the ability to perform <code>READ VOLUME</code> , <code>WRITE VOLUME</code> , and <code>APPLY TAG</code>
Function (including models)	Implies the ability to perform <code>EXECUTE</code> . For registered models, also implies <code>APPLY TAG</code> and <code>CREATE MODEL VERSION</code>
Feature	Implies the ability to perform <code>READ FEATURE</code>

The following table describes what `ALL PRIVILEGES` implies for non-data securable objects:

Object type	Implied privileges	
Connection	Implies the ability to perform <code>CREATE FOREIGN CATALOG</code> and <code>USE CONNECTION</code>	
External location	Implies the ability to perform <code>BROWSE</code> , <code>CREATE EXTERNAL TABLE</code> , <code>CREATE EXTERNAL VOLUME</code> , <code>CREATE FOREIGN SECURABLE</code> , <code>CREATE MANAGED STORAGE</code> , <code>READ FILES</code> , and <code>WRITE FILES</code> *	
External metadata	Implies the ability to perform <code>APPLY TAG</code> , <code>BROWSE</code> , and <code>MODIFY</code>	
Service credential	Implies the ability to perform <code>ACCESS</code> and <code>CREATE CONNECTION</code>	
Storage credential	Implies the ability to perform <code>CREATE EXTERNAL LOCATION</code> , <code>CREATE EXTERNAL TABLE</code> , <code>READ FILES</code> , and <code>WRITE FILES</code>	

*To prevent accidental data exfiltration or privilege escalation, `ALL PRIVILEGES` does not include the `EXTERNAL USE SCHEMA`, `EXTERNAL USE LOCATION`, `MANAGE`, or `READ METADATA` privileges.

When listing permissions using the Databricks API or with a `SHOW GRANTS` command for a user with `ALL PRIVILEGES`, only `ALL PRIVILEGES` is returned, not the individual implied privileges like `SELECT` or `MODIFY`.

When `ALL PRIVILEGES` is revoked, both the `ALL PRIVILEGES` grant and any individual privileges implied by it are removed. The `EXTERNAL USE SCHEMA`, `EXTERNAL USE LOCATION`, `MANAGE`, and `READ METADATA` privileges are not affected.

To maintain backward compatibility, `ALL PRIVILEGES` is evaluated at the time permissions checks are made. This means that as Databricks releases new privileges and securable objects, an existing `ALL PRIVILEGES` grant automatically includes any new privileges applicable to the securable object, and all new and existing child objects.

APPLY TAG

- **Applicable object types:** `EXTERNAL METADATA`, `FUNCTION` (registered models only), `MATERIALIZED VIEW`, `SERVICE`, `TABLE`, `VIEW`, `VOLUME`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to add and edit tags on a securable object. In addition:

- For a `table` or `view`, having `APPLY TAG` also enables column-level tagging.
- For a registered `model`, `APPLY TAG` also enables version-level tagging.

The user must also have `USE CATALOG` on the parent catalog and `USE SCHEMA` on the parent schema. `External metadata` objects are not contained in a catalog or schema, so these usage privileges do not apply.

To apply a `governed tag`, you must also have the `ASSIGN` privilege on the governed tag. See [Manage permissions on governed tags](#).  Ask Genie

ⓘ PUBLIC PREVIEW

Applying tags to [external metadata](#) objects is in Public Preview. SQL support for tagging external metadata requires Databricks Runtime 18.2 or above.

BROWSE

- **Applicable object types:** `CLEAN ROOM`, `EXTERNAL LOCATION`, `EXTERNAL METADATA`
- **Applicable container objects:** `CATALOG`

The `BROWSE` privilege is a special privilege that allows users to discover and view metadata about objects without granting access to the underlying data. Users with `BROWSE` can:

- See that an object exists
- View its name, description, and tags
- Request access to it

For data objects (tables, views, volumes, and functions), `BROWSE` can be granted at the catalog level only. It allows you to discover and view all objects within that catalog, but it won't explicitly appear as an inherited privilege when viewing permissions on schemas and child objects in Catalog Explorer.

For [external locations](#), [external metadata](#), and [clean rooms](#), you can grant `BROWSE` directly on the object itself.

Users with `BROWSE` do not need usage privileges like `USE CATALOG` or `USE SCHEMA` to discover and view metadata.

Databricks recommends granting `BROWSE` on [catalogs](#) to the `All account users` group to make data discoverable throughout your organization. This enables users to find data and request access without requiring administrators to grant permissions preemptively.



CREATE CATALOG

- **Applicable container objects: Unity Catalog metastore**

Allows a user to create a [catalog](#) in a Unity Catalog [metastore](#). To create a foreign catalog, you must also have the [CREATE FOREIGN CATALOG](#) privilege on the [connection](#) that contains the foreign catalog or on the [metastore](#).

CREATE CLEAN ROOM

- **Applicable container objects: Unity Catalog metastore**

Allows a user to create a [clean room](#) for securely collaborating on projects with other organizations without sharing underlying data.

CREATE CONNECTION

- **Applicable object types:** `SERVICE CREDENTIAL`
- **Applicable container objects: Unity Catalog metastore**

Allows a user to create a [connection](#) to an external database in a Lakehouse Federation scenario. To use a service credential to create a connection, the user must have this privilege on both the [metastore](#) and the [service credential](#).

CREATE EXTERNAL LOCATION

- **Applicable object types:** `STORAGE CREDENTIAL`
- **Applicable container objects: Unity Catalog metastore**

To create an [external location](#), the user must have this privilege on both the [metastore](#) and the [storage credential](#) that is being referenced in the external location.

CREATE EXTERNAL METADATA

- **Applicable container objects:** Unity Catalog metastore

Allows a user to create an [external metadata](#) object for use in custom lineage. To add lineage relationships to the external metadata object, the user must have the `MODIFY` privilege on the external metadata object, along with privileges on the Unity Catalog object that they are specifying the relationship with.

CREATE EXTERNAL TABLE

- **Applicable object types:** `EXTERNAL LOCATION`, `STORAGE CREDENTIAL`

Allows a user to create external tables directly in your cloud tenant using an [external location](#) or [storage credential](#). Databricks recommends granting this privilege on an external location rather than storage credential because it's scoped to a path, which allows more control over where users can create external tables in your cloud tenant.

CREATE EXTERNAL VOLUME

- **Applicable object types:** `EXTERNAL LOCATION`

Allows a user to create external volumes using an [external location](#).

CREATE FEATURE

ⓘ PUBLIC PREVIEW

The Feature securable object is in Public Preview.

- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to create a [feature](#) in a [schema](#) on which `CREATE FEATURE` is granted. Following the principle of least privilege, Databricks recommends granting `CREATE FEATURE` at the schema level. You can also grant `CREATE FEATURE` on a [catalog](#) to allow a user to create features in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and the `USE SCHEMA` privilege on the schema where the feature is created.

CREATE FOREIGN CATALOG

- **Applicable object types:** CONNECTION

Allows a user to create foreign catalogs using a [connection](#) to an external database in a Lakehouse Federation scenario.

CREATE FOREIGN SECURABLE

- **Applicable object types:** EXTERNAL LOCATION

Allows a user who is creating a foreign catalog to specify [authorized paths](#) that are covered by the [external location](#).

The user must also have `CREATE CATALOG` on the [metastore](#) and `CREATE FOREIGN CATALOG` on the [connection](#).

CREATE FUNCTION



- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to create a [function](#). Following the principle of least privilege, Databricks recommends granting `CREATE FUNCTION` at the schema level, which allows users to create functions in that schema. You can also grant `CREATE FUNCTION` on a [catalog](#) to allow a user to create [functions](#) in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and `USE SCHEMA` on the schema where the function is created.

CREATE MANAGED STORAGE

- **Applicable object types:** EXTERNAL LOCATION

Allows a user to configure a custom managed storage location within an [external location](#) when creating a [catalog](#) or [schema](#). When used during catalog creation, the specified location becomes the default managed storage for all schemas in that catalog (overriding the [metastore](#) default). When used during schema creation, the specified location applies to that schema only (overriding any catalog-level default).

CREATE MATERIALIZED VIEW

- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to create a [materialized view](#) in a [schema](#) on which `CREATE MATERIALIZED VIEW` is granted. Following the principle of least privilege, Databricks recommends granting `CREATE MATERIALIZED VIEW` at the schema level. You can also grant `CREATE MATERIALIZED VIEW` on a [catalog](#) to allow a user to create materialized views in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and `USE SCHEMA` on the schema where the materialized view is created.

CREATE MODEL

- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to create an MLflow registered [model](#) (which is a type of [function](#)) in a [schema](#) on which `CREATE MODEL` is granted. Following the principle of least privilege, Databricks recommends granting `CREATE MODEL` at the schema level. You can also grant `CREATE MODEL` on a [catalog](#) to allow a user to create registered models in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and `USE SCHEMA` on the schema where the model is created.

CREATE MODEL VERSION

- **Applicable object types:** MODEL

Allows a user to register a new version of an MLflow registered [model](#) (which is a type of [function](#)). Does not grant permission to execute, modify, or add tags to a model version.

The user must also have the `USE CATALOG` privilege on the parent catalog and `USE SCHEMA` on the parent schema.

CREATE SCHEMA

- **Applicable container objects:** CATALOG

Allows a user to create a [schema](#) in a [catalog](#) on which `CREATE SCHEMA` is granted. The user must also have the `USE CATALOG` privilege on the catalog.

CREATE SECRET

- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to create a secret in a [schema](#) on which `CREATE SECRET` is granted. The user must also have `USE CATALOG` on the catalog and `USE SCHEMA` on the schema where the secret is created. You can also grant `CREATE SECRET` on a [catalog](#) to allow a user to create secrets in any current or future schema in the catalog. For more information about secrets, see [Secrets in Unity Catalog](#).

CREATE SERVICE

- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to create a service in a [schema](#) on which `CREATE SERVICE` is granted. Services include [model services](#), [model provider services](#), [MCP services](#), and [agent services](#). Following the principle of least privilege, Databricks recommends granting `CREATE SERVICE` at the schema level. You can also grant `CREATE SERVICE` on a [catalog](#) to allow a user to create services in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and `USE SCHEMA` on the schema where the service is created.

CREATE SERVICE CREDENTIAL

- **Applicable container objects:** Unity Catalog metastore

Allows a user to create a [service credential](#) in a Unity Catalog [metastore](#).

CREATE STORAGE CREDENTIAL

- **Applicable container objects:** Unity Catalog metastore

Allows a user to create a [storage credential](#) in a Unity Catalog [metastore](#).

CREATE TABLE

- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to create a [table](#) or [view](#) in a [schema](#) on which `CREATE TABLE` is granted. Following the principle of least privilege, Databricks recommends granting `CREATE TABLE` at the schema level. You can also grant `CREATE TABLE` on a [catalog](#) to allow a user to create tables or views in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and the `USE SCHEMA` privilege on the schema where the table or view is created.

CREATE VOLUME

- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to create a [volume](#) in a [schema](#) on which `CREATE VOLUME` is granted. Following the principle of least privilege, Databricks recommends granting `CREATE VOLUME` at the schema level. You can also grant `CREATE VOLUME` on a [catalog](#) to allow a user to create [volumes](#) in any existing or future schema in the catalog.

The user must also have the `USE CATALOG` privilege on the catalog and the `USE SCHEMA` privilege on the schema where the volume is created.

DELETE



BETA



This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

- **Applicable object types:** `TABLE`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to delete existing data from a [table](#). `DELETE` is one of the fine-grained DML privileges, a least-privilege alternative to `MODIFY`. See [Fine-grained DML privileges](#).

EXECUTE

- **Applicable object types:** `FUNCTION`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to invoke a [function](#) or load a registered [model](#) for inference. For functions, `EXECUTE` also grants the ability to view the function definition and metadata. For registered models, `EXECUTE` also grants the ability to view metadata for all model versions and download model files.

Following the principle of least privilege, Databricks recommends granting `EXECUTE` on individual functions. You can also grant `EXECUTE` on a [schema](#) or [catalog](#) to allow a user to execute all current and future functions within that schema or catalog.

The user must also have `USE CATALOG` on the parent catalog and `USE SCHEMA` on the parent schema.

EXECUTE CLEAN ROOM TASK

- **Applicable object types:** `CLEAN ROOM`

Allows a user to run tasks (notebooks) in a [clean room](#). Also enables the user to view clean room details.



EXTERNAL USE LOCATION

- **Applicable object types:** `EXTERNAL LOCATION`

Allows a user to obtain a temporary credential to access an [external location](#) from an external processing engine using the Unity Catalog open APIs or Apache Spark.

To avoid accidental data exfiltration, `ALL PRIVILEGES` does not include the `EXTERNAL USE LOCATION` privilege, and external location owners do not have this privilege by default. This means that only users with the `MANAGE` privilege on the external location can grant this privilege.

See [Enable external data access to Unity Catalog](#).

EXTERNAL USE SCHEMA

ⓘ PREVIEW

This feature is in [Public Preview](#).

- **Applicable container objects:** `SCHEMA`

Allows a user to be granted a temporary credential to access Unity Catalog tables from an external processing engine using the Unity Catalog open APIs or Iceberg REST APIs.

To avoid accidental data exfiltration, `ALL PRIVILEGES` does not include the `EXTERNAL USE SCHEMA` privilege, and schema owners do not have this privilege by default. Only the [catalog](#) owner can grant this privilege.

See [Enable external data access to Unity Catalog](#).



INSERT

ⓘ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

- **Applicable object types:** `TABLE`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to add new data to a [table](#). `INSERT` is one of the fine-grained DML privileges, a least-privilege alternative to `MODIFY`. See [Fine-grained DML privileges](#).

MANAGE

- **Applicable object types:** `CLEAN ROOM`, `CONNECTION`, `EXTERNAL LOCATION`, `EXTERNAL METADATA`, `FEATURE`, `FUNCTION (including models)`, `MATERIALIZED VIEW`, `SERVICE CREDENTIAL`, `STORAGE CREDENTIAL`, `TABLE`, `VIEW`, `VOLUME`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to manage privileges on, transfer ownership of, and delete an object without being the owner. `MANAGE` is similar to object ownership, but there are some important differences. See [Ownership versus the `MANAGE` privilege](#).

Users with `MANAGE` are not automatically granted all privileges on the object. They must be granted each specific privilege separately, but users with `MANAGE` can explicitly grant themselves these privileges.

To exercise `MANAGE`, the user must also have the required usage privileges on the object's parent containers. `MANAGE` doesn't require a usage privilege on the same level that it's granted. See [Usage privilege requirements for `MANAGE`](#).



If `MANAGE` is granted on a container object, the user also gets `MANAGE` on all child objects. For example, granting `MANAGE` on a `catalog` also explicitly grants `MANAGE` on all child schemas and tables.

`ALL PRIVILEGES` does not include the `MANAGE` privilege.

MANAGE ACCESS CONTROL

NOTE

`MANAGE ACCESS CONTROL` can be *denied*, through an [ABAC DENY policy](#), but not granted. For details, see [ABAC DENY policies \(Beta\)](#).

Allows a user to manage access control on an object without granting the full `MANAGE` privilege. A [child privilege](#) of `MANAGE`, it grants a subset of `MANAGE`'s capabilities, including:

- Granting and revoking privileges
- Transferring ownership
- Creating and managing [attribute-based access control \(ABAC\)](#) policies
- Configuring [access request destinations](#)
- Managing [workspace-catalog bindings](#), when granted on a catalog

MANAGE ALLOWLIST

- **Applicable container objects: Unity Catalog metastore**

Allows a user to add, modify, and remove entries in the allowlist that controls which init scripts, JAR files, and Maven coordinates can run on Unity Catalog-enabled clusters configured with standard access mode. By default, the allowlist is empty.  **Ask Genie**

Because users with `MANAGE ALLOWLIST` can control what code runs on standard access mode compute, Databricks recommends granting this privilege to metastore admins and trusted platform administrators only.

See [Allowlist libraries and init scripts on compute with standard access mode \(formerly shared access mode\)](#).

MODIFY

- **Applicable object types:** `EXTERNAL METADATA`, `TABLE`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

When applied to a [table](#), allows a user to insert, update, and delete data in the table. The user must also have `SELECT` on the table, `USE SCHEMA` on the parent schema, and `USE CATALOG` on the parent catalog.

Due to [privilege inheritance](#), you can grant `MODIFY` on a [schema](#) to automatically grant `MODIFY` on all current and future tables in the schema. Similarly, you can grant `MODIFY` on a [catalog](#) to automatically grant `MODIFY` on all current and future tables in the catalog.

When applied to an [external metadata](#) object, allows a user to add lineage relationships to that object.

NOTE

`MODIFY` cannot be granted on a [foreign table](#) because foreign tables are read-only.

MODIFY CLEAN ROOM

- **Applicable object types:** `CLEAN ROOM`

Allows a user to update a [clean room](#), which includes adding and removing data assets, adding and removing notebooks, and updating comments. Also enables the user to view clean room details.

READ FEATURE

⚠ PUBLIC PREVIEW

The Feature securable object is in Public Preview.

- **Applicable object types:** `FEATURE`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to read a [feature](#)'s metadata and materialized data and to use the feature for model inference and training. The user must also have `USE CATALOG` on the parent catalog and `USE SCHEMA` on the parent schema.

Due to [privilege inheritance](#), you can grant `READ FEATURE` on a [schema](#) to automatically grant `READ FEATURE` on all current and future features in the schema. Similarly, you can grant `READ FEATURE` on a [catalog](#) to automatically grant `READ FEATURE` on all current and future features in the catalog.

READ FILES

- **Applicable object types:** `EXTERNAL LOCATION`

Allows a user to read files directly from cloud object storage configured as an [external location](#). Databricks recommends against reading files directly from cloud object storage. Instead, manage read access to data in cloud object storage using [volumes](#) and the `READ VOLUME` privilege. See [External locations](#).

`READ FILES` is also required for write operations on external locations. Any principal that has the `WRITE FILES` privilege only on an external location receives a `PERMISSION_DENIED` error when attempting to write files. See [WRITE FILES](#).

READ METADATA

- **Applicable object types:** `CLEAN ROOM`, `CONNECTION`, `EXTERNAL LOCATION`, `EXTERNAL METADATA`, `FEATURE`, `FUNCTION` (including models), `MATERIALIZED VIEW`, `SERVICE`, `SERVICE CREDENTIAL`, `STORAGE CREDENTIAL`, `TABLE`, `VIEW`, `VOLUME`
- **Applicable container objects:** `Unity Catalog metastore`, `SCHEMA`, `CATALOG`

Allows a user to view all of an object's metadata without granting the ability to modify the object or read its data. `READ METADATA` provides read-only visibility into the same metadata available to the object's owner or a user with `MANAGE`, including security-sensitive information that `BROWSE` doesn't expose. This includes, but is not limited to:

- Privilege grants
- [Row filters and column masks](#)
- [Attribute-based access control \(ABAC\) policies](#)
- [View and function definitions](#)
- Credential details, such as storage credential names and IDs

`READ METADATA` is intended for users who inspect and debug access controls, such as security auditors, data governance teams, and site reliability engineers (SREs). Because `READ METADATA` exposes sensitive metadata, Databricks recommends granting this privilege only to trusted principals. To let users discover objects and view basic metadata without exposing this sensitive information, grant `BROWSE` instead. See [BROWSE](#).

`READ METADATA` is a [child privilege](#) of `MANAGE`. Like `MANAGE`, it is not included in `ALL PRIVILEGES` and must be granted explicitly. See [MANAGE](#).

To exercise `READ METADATA`, the user must also have the required usage privileges on the object's parent containers. `READ METADATA` doesn't require a usage privilege on the same level that it's granted. No usage privileges are required when `READ METADATA` is granted on the [metastore](#) or on a [catalog](#). See [Usage privilege requirements for MANAGE](#).

Due to [privilege inheritance](#), granting `READ METADATA` on a [catalog](#) or [schema](#) automatically grants it on all current and future objects they contain. Unlike other metastore-level privileges, `READ METADATA` granted on the [metastore](#) also inherits down to all objects in the metastore.

READ SECRET

- **Applicable object types:** `SECRET`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to retrieve a secret value from a notebook or job, using `dbutils` or the Unity Catalog REST API. The user must also have `USE SCHEMA` on the parent schema and `USE CATALOG` on the parent catalog.

Due to [privilege inheritance](#), you can grant `READ SECRET` on a [schema](#) or [catalog](#) to automatically grant it on all current and future secrets in the schema or catalog. For more information about secrets, see [Secrets in Unity Catalog](#).

READ VOLUME

- **Applicable object types:** `VOLUME`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to read files and directories stored inside a [volume](#). The user must also have `USE SCHEMA` on the parent schema and `USE CATALOG` on the parent catalog.

Due to [privilege inheritance](#), you can grant `READ VOLUME` on a [schema](#) to automatically grant `READ VOLUME` on all current and future volumes in the schema. Similarly, you can grant `READ VOLUME` on a [catalog](#) to automatically grant `READ VOLUME` on all current and future volumes in the catalog.

REFERENCE SECRET

- **Applicable object types:** `SECRET`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a Unity Catalog object to reference a secret by name so that the object can use the secret value at runtime without exposing it to the user. The user must also have `USE SCHEMA` on the parent schema and `USE CATALOG` on the parent catalog.

Depending on the object, referencing a secret requires either `REFERENCE SECRET` or `READ SECRET`. For example, creating or updating a [connection](#) that references a secret requires `READ SECRET`.

Due to [privilege inheritance](#), you can grant `REFERENCE SECRET` on a [schema](#) or [catalog](#) to automatically grant it on all current and future secrets in the schema or catalog. For more information about secrets, see [Secrets in Unity Catalog](#).

REFRESH

- **Applicable object types:** `MATERIALIZED VIEW`
- **Applicable container objects:** `SCHEMA`, `CATALOG`

Allows a user to refresh a materialized view. The user must also have `USE SCHEMA` on the parent schema and `USE CATALOG` on the parent catalog.

Due to [privilege inheritance](#), you can grant `REFRESH` on a [schema](#) to automatically grant `REFRESH` on all current and future materialized views in the schema. Similarly, you can grant `REFRESH` on a [catalog](#) to automatically grant `REFRESH` on all current and

future materialized views in the catalog.

SELECT

- **Applicable object types:** MATERIALIZED VIEW, SHARE, TABLE, VIEW
- **Applicable container objects:** SCHEMA, CATALOG

When applied to a [table](#), [view](#), or materialized view, allows a user to select from the object. The user must also have `USE CATALOG` on the parent catalog and `USE SCHEMA` on the parent schema. When applied to a [share](#), allows a [recipient](#) to select from the share.

Due to [privilege inheritance](#), you can grant `SELECT` on a [schema](#) to automatically grant `SELECT` on all current and future tables and views in the schema. Similarly, you can grant `SELECT` on a [catalog](#) to automatically grant `SELECT` on all current and future tables and views in the catalog.

UPDATE

ⓘ BETA

This feature is in [Beta](#). Workspace admins can control access to this feature from the **Previews** page. See [Manage Databricks previews](#).

- **Applicable object types:** TABLE
- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to modify existing data in a [table](#). `UPDATE` is one of the fine-grained DML privileges, a least-privilege alternative to `MODIFY`. See [Fine-grained DML privileges](#).



USE CATALOG

- **Applicable container objects:** CATALOG

USE CATALOG is a [usage privilege](#). Generally, users need this privilege to interact with any object within the [catalog](#). USE CATALOG does not grant access to the catalog itself or to any specific objects within it.

For example, to read from a table, a user needs `SELECT` on the table, `USE CATALOG` on the parent catalog, and `USE SCHEMA` on the parent schema.

USE CATALOG also provides an important access control boundary for catalog owners. Even if a table owner grants `SELECT` on a table to another user, that user cannot access the table unless they also have `USE CATALOG` on the parent catalog. Because only catalog owners or users with `MANAGE` on the catalog can grant `USE CATALOG`, catalog owners retain control over which users can access their objects, regardless of what individual table or schema owners grant.

USE CATALOG is not required to discover or read object metadata if the user has the `BROWSE` privilege on that catalog.

USE CONNECTION

- **Applicable object types:** CONNECTION

Allows a user to list and view details about [connections](#) to external databases in a [Lakehouse Federation](#) scenario. `USE CONNECTION` is also required to use the [remote_query function](#) to run SQL queries directly on external databases.

USE SCHEMA

- **Applicable container objects:** SCHEMA, CATALOG

`USE SCHEMA` is a [usage privilege](#). Generally, users need this privilege to interact with any object within the [schema](#). `USE SCHEMA` does not grant access to the schema itself or to any specific objects within it.

For example, to read from a table, a user needs `SELECT` on the table, `USE SCHEMA` on the parent schema, and `USE CATALOG` on the parent catalog.

`USE SCHEMA` also provides an important access control boundary for schema owners. Even if a table owner grants `SELECT` on a table to another user, that user cannot access the table unless they also have `USE SCHEMA` on the parent schema. Because only schema owners or users with `MANAGE` on the schema can grant `USE SCHEMA`, schema owners retain control over which users can access their objects, regardless of what individual table owners grant.

Due to [privilege inheritance](#), you can grant `USE SCHEMA` on a [catalog](#) to automatically grant `USE SCHEMA` on all current and future schemas in the catalog.

`USE SCHEMA` is not required to discover or read object metadata if the user has the [BROWSE](#) privilege on the parent catalog.

WRITE FILES

- **Applicable object types:** `EXTERNAL LOCATION`

Allows a user to write files directly to cloud object storage configured as an [external location](#). Databricks recommends against writing files directly to cloud object storage. Instead, manage write access to data in cloud object storage using [volumes](#) and the `WRITE VOLUME` privilege. For more guidance, see [Managed and external volumes](#).

NOTE

`WRITE FILES` requires `READ FILES` to also be granted on the same external location. Write operations on cloud object storage involve metadata checks and path validation that require read access.



WRITE SECRET

- **Applicable object types:** SECRET
- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to update a secret value. The user must also have `USE SCHEMA` on the parent schema and `USE CATALOG` on the parent catalog.

Due to [privilege inheritance](#), you can grant `WRITE SECRET` on a [schema](#) or [catalog](#) to automatically grant it on all current and future secrets in the schema or catalog. For more information about secrets, see [Secrets in Unity Catalog](#).

WRITE VOLUME

- **Applicable object types:** VOLUME
- **Applicable container objects:** SCHEMA, CATALOG

Allows a user to add, remove, or modify files and directories stored inside a [volume](#). The user must also have `USE CATALOG` on the parent catalog and `USE SCHEMA` on the parent schema.

Due to [privilege inheritance](#), you can grant `WRITE VOLUME` on a [schema](#) to automatically grant `WRITE VOLUME` on all current and future volumes in the schema. Similarly, you can grant `WRITE VOLUME` on a [catalog](#) to automatically grant `WRITE VOLUME` on all current and future volumes in the catalog.

Privileges that apply only to OpenSharing or Databricks Marketplace



This section provides details about the privileges that apply only to OpenSharing.

CREATE PROVIDER

- **Applicable container objects: Unity Catalog metastore**

Allows a user to create an OpenSharing [provider](#) object in the [metastore](#). A provider identifies an organization or group of users that shares data using OpenSharing. Provider objects are created by a user in the [recipient's](#) Databricks account. See [What is OpenSharing?](#).

CREATE RECIPIENT

- **Applicable container objects: Unity Catalog metastore**

Allows a user to create an OpenSharing [recipient](#) object in the [metastore](#). A recipient identifies an organization or group of users that receives shared data using OpenSharing. Recipient objects are created by a user in the [provider's](#) Databricks account. See [What is OpenSharing?](#).

CREATE SHARE

- **Applicable container objects: Unity Catalog metastore**

Allows a user to create a [share](#) in the [metastore](#). A share is a logical grouping of [tables](#) and other assets that a [provider](#) intends to share using OpenSharing.

SET SHARE PERMISSION

- **Applicable container objects: Unity Catalog metastore**

In OpenSharing, `SET SHARE PERMISSION` allows a [provider](#) user to set permissions on a [share](#), including granting [recipient](#) access and transferring ownership. To grant a [recipient](#) access to a [share](#), the user must also have `USE SHARE`, and either `USE RECIPIENT` or ownership of the [recipient](#) object. To transfer ownership of a [share](#), the user must also have `USE SHARE`.

USE MARKETPLACE ASSETS

- **Applicable container objects: Unity Catalog metastore**

Enabled by default for all Unity Catalog [metastores](#). In Databricks Marketplace, `USE MARKETPLACE ASSETS` allows a user to get or request access to data products in Marketplace listings. It also allows a user to access the read-only [catalog](#) that is created when a [provider](#) shares a data product.

Without this privilege, users must have the `CREATE CATALOG` and `USE PROVIDER` privileges, or the metastore admin role. Granting `USE MARKETPLACE ASSETS` instead allows administrators to limit the number of users with those more powerful privileges.

USE PROVIDER

- **Applicable container objects: Unity Catalog metastore**

In OpenSharing, `USE PROVIDER` allows a [recipient](#) user to view all [providers](#) in the [metastore](#) and their associated [shares](#) (read-only). Combined with `CREATE CATALOG`, `USE PROVIDER` also allows a [recipient](#) user who is not a metastore admin to mount a [share](#) as a [catalog](#). This allows administrators to limit the number of users with the metastore admin role.

USE RECIPIENT

- **Applicable container objects: Unity Catalog metastore**

In OpenSharing, `USE RECIPIENT` allows a `provider` user to view all `recipients` in the `metastore` (read-only), including recipient details, authentication status, and the `shares` the `provider` has shared with each `recipient`. A `provider` user does not need to be a metastore admin to use this privilege.

In [Databricks Marketplace](#), `USE RECIPIENT` allows `provider` users to view listings and consumer requests in the Provider console.

USE SHARE

- **Applicable container objects: Unity Catalog metastore**

In OpenSharing, `USE SHARE` allows a `provider` user to view all `shares` in the `metastore` (read-only), including the assets (`tables` and notebooks) in each `share` and the `share's recipients`. A `provider` user does not need to be a metastore admin to use this privilege.

In [Databricks Marketplace](#), `USE SHARE` allows `provider` users to view details about the data shared in a listing.